

Assignment: Applying the Decorator Pattern in Software Design

Course: CP353002 Principles of software Design and Development

Topic: Structural Design Patterns - Decorator Pattern

Total Score: 100 Points

Learning Outcomes

เมื่อจบงานนี้นักศึกษาจะสามารถ

1. วิเคราะห์ปัญหาที่เหมาะสมกับ Decorator Pattern
2. ออกแบบ UML Class Diagram สำหรับ Decorator Pattern
3. พัฒนาระบบโดยใช้หลัก Composition over Inheritance
4. อธิบายความสัมพันธ์ระหว่าง Decorator Pattern และ Open-Closed Principle
5. เปรียบเทียบ Decorator Pattern กับการใช้ Inheritance แบบดั้งเดิม

Case Study

บริษัทพัฒนาซอฟต์แวร์ได้รับมอบหมายให้ออกแบบระบบสั่งกาแฟสำหรับร้านกาแฟแห่งหนึ่ง ลูกค้าสามารถเลือกเครื่องดื่มหลักและเพิ่มส่วนประกอบเสริมได้อย่างอิสระ

Base Products

- Espresso (\$2.00)
- Americano (\$2.50)

Optional Features

- Milk (+\$0.50)
- Sugar (+\$0.20)
- Whipped Cream (+\$0.70)

ในอนาคตอาจมี Add-on ใหม่เพิ่มเติม เช่น

- Caramel
- Chocolate
- Honey
- Ice Cream

ดังนั้น ระบบต้องสามารถขยายความสามารถได้โดยไม่ต้องแก้ไขคลาสเดิม

Tasks

Part 1: Problem Analysis (15 คะแนน)

จงตอบคำถามต่อไปนี้

1.1 หากใช้ Inheritance เพียงอย่างเดียว จะเกิดปัญหาอะไรขึ้นเมื่อมีการเพิ่ม Add-on จำนวนมาก

ตอบ ปัญหาหลักคือจำนวนคลาสและจำนวน Combination ของคุณสมบัติเพิ่มขึ้นอย่างรวดเร็ว หากมี Base Coffee หลายชนิดและมี Add-on หลายรายการ อาจต้องสร้าง Subclass สำหรับแต่ละ Combination ทำให้โครงสร้างระบบซับซ้อน ดูแลรักษายาก และเพิ่มความเสี่ยงในการแก้ไขคลาสเดิม

1.2 เพราะเหตุใด Decorator Pattern จึงเหมาะสมกับระบบนี้

ตอบ เพราะสามารถเพิ่มส่วนประกอบเสริมให้กับเครื่องดื่มได้อย่างอิสระ และสามารถเพิ่ม Decorator หลายชั้นได้ โดยไม่จำเป็นต้องสร้างคลาสใหม่สำหรับทุก Combination ของเครื่องดื่มและ Add-on

1.3 Decorator Pattern สนับสนุน Open-Closed Principle อย่างไร

ตอบ ช่วยให้ระบบสามารถเพิ่มความสามารถใหม่ผ่านการสร้าง Concrete Decorator ใหม่ โดยไม่จำเป็นต้องแก้ไขคลาส Coffee, Espresso หรือ Americano เดิม ดังนั้นระบบจึงสามารถเปิดให้ขยายความสามารถได้ แต่ลดความจำเป็นในการแก้ไขโค้ดเดิม

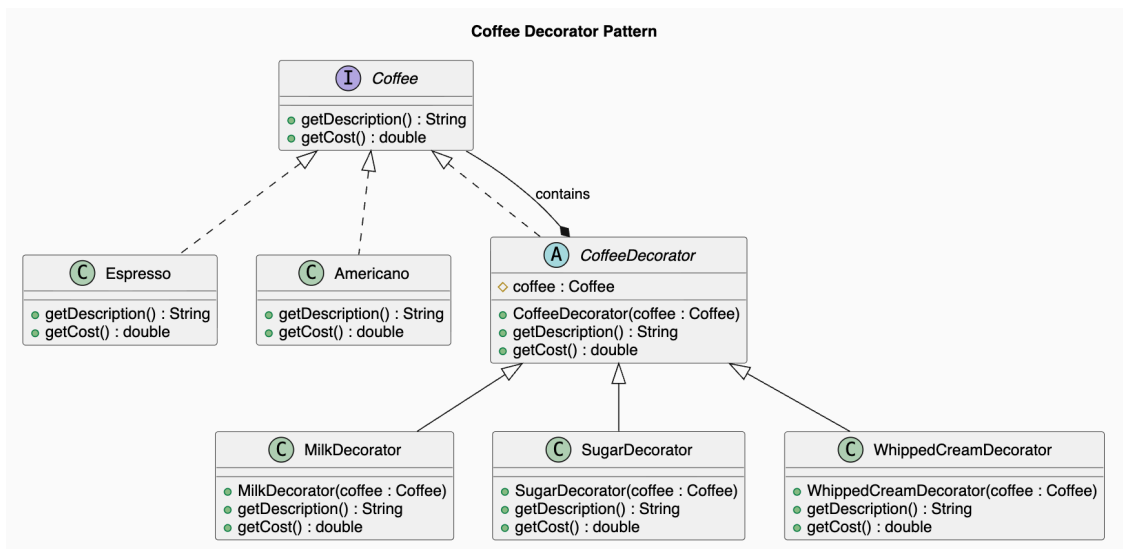
Part 2: UML Design (20 คะแนน)

ออกแบบ UML Class Diagram โดยต้องประกอบด้วย

1. Component ได้แก่ Coffee
2. Concrete Components ได้แก่ Espresso และ Americano
3. Decorator ได้แก่ CoffeeDecorator
4. Concrete Decorators ได้แก่ MilkDecorator, SugarDecorator, WhippedCreamDecorator

โดยแสดง

- Inheritance Relationship
- Association/Composition Relationship
- Method Signatures



Part 3: Implementation (40 คะแนน)

พัฒนาโปรแกรมด้วยภาษา Java โดยใช้ Decorator Pattern

Requirement 1

สร้าง Interface Coffee

Requirement 2

สร้างคลาส Espresso, Americano

Requirement 3

สร้าง Abstract Decorator ชื่อคลาสชื่อ CoffeeDecorator

Requirement 4

สร้างคลาสที่ทำหน้าที่เป็น Decorators ได้แก่คลาสต่อไปนี้

1. MilkDecorator
2. SugarDecorator
3. WhippedCreamDecorator

Requirement 5

รองรับการเพิ่ม Decorator หลายชั้น

ตัวอย่าง

```
Coffee order = new WhippedCreamDecorator(
    new SugarDecorator(
        new MilkDecorator(
            new Americano())));
```

โค้ด :

```
public interface Coffee{
    String getDescription();
    double getCost();
}
```

```
public class Espresso implements Coffee{
    @Override
    public String getDescription() {
        return "Espresso";
    }
    @Override
    public double getCost() {
        return 2.00;
    }
}
```

```
public class Americano implements Coffee{
    @Override
    public String getDescription() {
        return "Americano";
    }
    @Override
    public double getCost() {
        return 2.50;
    }
}
```

```
public abstract class CoffeeDecorator implements Coffee{
    protected Coffee coffee;

    public CoffeeDecorator(Coffee coffee) {
        this.coffee = coffee;
    }
    @Override
    public String getDescription() {
        return coffee.getDescription();
    }
    @Override
    public double getCost() {
        return coffee.getCost();
    }
}
```

```

    }
}

```

```

public class MilkDecorator extends CoffeeDecorator {

```

```

    public MilkDecorator(Coffee coffee) {
        super(coffee);
    }
    @Override
    public String getDescription() {
        return coffee.getDescription() + " + Milk";
    }
    @Override
    public double getCost() {
        return coffee.getCost() + 0.50;
    }
}

```

```

public class SugarDecorator extends CoffeeDecorator {

```

```

    public SugarDecorator(Coffee coffee) {
        super(coffee);
    }
    @Override
    public String getDescription() {
        return coffee.getDescription() + " + Sugar";
    }
    @Override
    public double getCost() {
        return coffee.getCost() + 0.20;
    }
}

```

```

public class WhippedCreamDecorator extends CoffeeDecorator {

```

```

    public WhippedCreamDecorator(Coffee coffee) {
        super(coffee);
    }
    @Override
    public String getDescription() {
        return coffee.getDescription() + " + Whipped Cream";
    }
    @Override
    public double getCost() {
        return coffee.getCost() + 0.70;
    }
}

```

```

public class Main {
    public static void main(String[] args) {

        //Test Case 1
        Coffee coffee1 = new Espresso();
        System.out.println("Test Case 1");
        System.out.println("Configuration: Espresso");
        System.out.println("Description: " + coffee1.getDescription());
        System.out.printf("Total Cost: $%.2f%n", coffee1.getCost());
        System.out.println();

        //Test Case 2
        Coffee coffee2 = new MilkDecorator(new Espresso());
        System.out.println("Test Case 2");
        System.out.println("Configuration: Espresso + Milk");
        System.out.println("Description: " + coffee2.getDescription());
        System.out.printf("Total Cost: $%.2f%n", coffee2.getCost());
        System.out.println();

        //Test Case 3
        Coffee coffee3 = new WhippedCreamDecorator(new SugarDecorator(new
MilkDecorator(new Americano())));
        System.out.println("Test Case 3");
        System.out.println("Configuration: Americano + Milk + Sugar + Whipped Cream");
        System.out.println("Description: " + coffee3.getDescription());
        System.out.printf("Total Cost: $%.2f%n", coffee3.getCost());
    }
}

```

Part 4: Testing and Discussion (15 คะแนน)

ทดสอบอย่างน้อย 3 กรณี

Test Case 1

เลือกกาแฟ เป็น Espresso

Test Case 2

เลือกกาแฟแบบ Espresso + Milk

Test Case 3

เลือกกาแฟแบบ Americano + Milk + Sugar + Whipped Cream

สำหรับแต่ละกรณีให้แสดง

- Configuration
- Description
- Total Cost

ผลลัพธ์:

```

Test Case 1
Configuration: Espresso
Description: Espresso
Total Cost: $2.00

Test Case 2
Configuration: Espresso + Milk
Description: Espresso + Milk
Total Cost: $2.50

Test Case 3
Configuration: Americano + Milk + Sugar + Whipped Cream
Description: Americano + Milk + Sugar + Whipped Cream
Total Cost: $3.90

```

Part 5: Design Reflection (10 คะแนน)

ตอบคำถามต่อไปนี้

5.1 Decorator Pattern แตกต่างจาก Strategy Pattern อย่างไร

ตอบ

- Decorator Pattern ใช้สำหรับเพิ่มความสามารถหรือพฤติกรรมให้กับ Object โดยการห่อ Object เดิมด้วย Decorator หลายชั้น เช่น การเพิ่ม Milk, Sugar และ Whipped Cream ให้กับ Coffee
- Strategy Pattern ใช้สำหรับเลือกหรือเปลี่ยน Algorithm หรือวิธีการทำงานของ Object เช่น การเลือกวิธีคำนวณหรือวิธีการประมวลผลที่แตกต่างกัน

5.2 หากมี Add-on ใหม่เพิ่มขึ้น 20 รายการ ระบบที่ใช้ Decorator Pattern จะมีข้อดีอย่างไร

ตอบ สามารถสร้าง Concrete Decorator เพิ่มสำหรับ Add-on แต่ละรายการได้ โดยไม่จำเป็นต้องแก้ไข Coffee หรือ Concrete Component เดิม และสามารถนำ Decorator เหล่านั้นมาประกอบกันได้หลายรูปแบบ ทำให้ระบบสามารถรองรับการขยาย Add-on ได้ง่ายกว่าแนวทางที่สร้าง Subclass สำหรับทุก Combination

5.3 หลักการ "Composition over Inheritance" ปรากฏอยู่ในส่วนใดของงานนี้

ตอบ

```

public abstract class CoffeeDecorator implements Coffee {
    protected Coffee coffee;

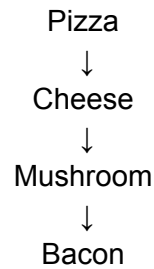
    public CoffeeDecorator(Coffee coffee) {
        this.coffee = coffee;
    }
}

```

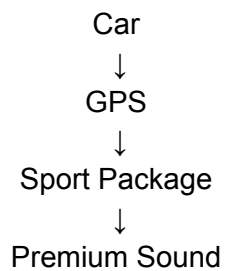
5.4 จงยกตัวอย่างระบบจริงที่สามารถใช้ Decorator Pattern ได้อีกอย่างน้อย 2 ระบบ

ตอบ

1. ระบบสั่งพิซซ่า สามารถเพิ่ม Topping หลายอย่างได้



2. ระบบแต่งรถ สามารถเพิ่ม Feature ให้รถได้หลายอย่าง



link git: <https://github.com/Rossarin289-4/DecoratorPattern.git>

Deliverables

นักศึกษาต้องส่ง

1. Source Code (.java)
2. UML Diagram (.png หรือ .pdf)
3. Test Results พร้อมภาพหน้าจอ
4. รายงาน (2-3 หน้า)

5. Assessment Rubric (100 คะแนน)

หัวข้อ	คะแนน
Problem Analysis	15
UML Design	20
Implementation	40
Testing	15
Design Reflection	10
รวม	100